机器人学上机操作实验报告

班级	自动化 3 班
姓名	卞常灿
学号	2023312216
所在学院	智能科学与工程学院
指导教师	杜惠
实验日期	2026 年 4 月 24 日
实验成绩	

编写：杜惠

哈尔滨工业大学（深圳）

1. 机器人逆解的推导过程，并给出Matlab代码（需要使用旋量方法）

1.1 旋量法（指数积）逆解推导过程

本实验对象为标准六轴工业机器人（满足腕部三轴交于一点的几何特征）。采用基于旋量理论（Twist）的指数积（POE）公式结合 Paden-Kahan 子问题进行逆运动学推导。具体步骤如下：

（1）定义运动旋量与初始位姿

根据机器人连杆长度参数（$L_1 = 491, L_2 = 450, L_3 = 450, L_4 = 84$），建立各关节的运动旋量 $\xi_i = [v_i, \omega_i]^T (i = 1...6)$。

根据初始状态，定义机器人零位参考位姿为 g_0（末端竖直向上，但工具坐标系绕Z轴旋转了-180度）。

（2）位姿解耦与初始状态剥离

设目标齐次变换矩阵为 T。为了求取纯关节旋转产生的位姿变换，首先剥离初始位姿 g_0，得到相对变换矩阵 $g_1 = T \cdot g_0^{-1}$。

（3）基于 **Paden-Kahan** 子问题 **1** 求解 θ_1

提取机器人零位下的腕部中心点（4、5、6轴交点）为 r_w。由于后三个关节的旋转不改变该点的位置，因此目标状态下腕部中心点的位置 $p_{target} = g_1 \cdot r_w$。

将 p_{target} 投影到基坐标系的 $X - Y$ 平面内，由于 ξ_2, ξ_3 的旋转轴平行于 Y 轴，不改变 Y 坐标关系，据此可直接利用反正切函数求解出 θ_1 的正反两组解。

（4）基于 **Paden-Kahan** 子问题 **3** 求解 θ_2 和 θ_3

利用已求得的 θ_1，通过旋量的指数映射 $e^{-\hat{\xi}_1 \theta_1}$ 将目标腕点 p_{target} 反向旋转回第一轴的 $X - Z$ 平面内，得到点 p'。

此时，θ_2 和 θ_3 构成了一个标准的平面两连杆问题。利用点 p' 相对旋量原点的坐标投影，结合余弦定理求出 θ_3（对应“肘部朝上”和“肘部朝下”两种构型）。随后通过几何夹角关系求解出 θ_2。

（5）球面腕部欧拉角反解求解 $\theta_4, \theta_5, \theta_6$

利用已求得的前三轴角度，计算前三轴的累积齐次变换 $g_{123} = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} e^{\hat{\xi}_3 \theta_3}$。

将其从总变换 g_1 中剥离，提取出仅由后三轴产生的纯旋转变换 $g_{456} = g_{123}^{-1} \cdot g_1$。

由于后三轴的旋转等效于 $Z-Y-Z$ 欧拉角，提取其 3×3 旋转矩阵 R_{456} 后，通过解析矩阵元素直接反解出 $\theta_4,\theta_5,\theta_6$（包含腕部“翻转”与“不翻转”两种构型）。当 θ_5 接近 0 或 π 时，进行奇异点（万向锁）的降阶处理。

最终交叉组合上述各步骤的多重解，可得到最多 8 组有效的逆运动学解析解。

1.2 Matlab 逆解代码实现

```
% AUTHOR :
%
% ABSTRACT: 基于纯旋量法(Paden-Kahan子问题)的六轴机器人逆运动学 (求全8组解)
%
% INPUT: config            机器人杆长信息
%        T                 机器人末端姿态矩阵，4x4矩阵
%
% OUTPUT: theta_sols       关节转角矩阵，6XN，每一列是一组解，单位rad
%
function theta_sols = Ikine6s_all(T, config)
    L1 = config(1); L2 = config(2); L3 = config(3); L4 = config(4);

    % --- 1. 旋量与基准位姿定义 ---
    g0 = [ -1,  0,  0,    0;
            0, -1,  0,    0;
            0,  0,  1, 1475;
            0,  0,  0,    1];

    Xi = [ 0,   -L1,  -(L1+L2),  0,  -(L1+L2+L3),  0;
           0,    0,     0,       0,    0,          0;
           0,    0,     0,       0,    0,          0;
           0,    0,     0,       0,    0,          0;
           0,    1,     1,       0,    1,          0;
           1,    0,     0,       1,    0,          1];

    g1 = T / g0;
    rw = [0; 0; L1+L2+L3; 1];
    p_target = g1 * rw;
    px = p_target(1);
    py = p_target(2);

    theta_sols = [];

    % --- 2. Paden-Kahan 子问题 1 求解 theta1 (2个解) ---
    th1_1 = atan2(py, px);
    th1_2 = th1_1 + pi;
    if th1_2 > pi, th1_2 = th1_2 - 2*pi; end
    theta1_opts = [th1_1, th1_1, th1_2, th1_2]; % 配合后续循环排列
```

```
    % --- 3. 循环求解后续关节 ---
    for i = 1:4
        th1 = theta1_opts(i);

        % Paden-Kahan 子问题 3 求解 theta2, theta3
        exp_minus_xi1 = twist_exp(-Xi(:,1), th1);
        p_prime = exp_minus_xi1 * p_target;

        u_x = p_prime(1);
        u_z = p_prime(3);

        r = u_x;
        s = u_z - L1;

        D = (r^2 + s^2 - L2^2 - L3^2) / (2 * L2 * L3);
        if abs(D) > 1, continue; end % 超出工作空间跳过

        % 根据循环索引分配 Elbow Up / Elbow Down
        if mod(i, 2) == 1
            th3 = atan2(-sqrt(1 - D^2), D);
        else
            th3 = atan2(sqrt(1 - D^2), D);
        end

        th2 = atan2(r, s) - atan2(L3 * sin(th3), L2 + L3 * cos(th3));

        % 提取纯旋转求球面腕部欧拉角 theta4, theta5, theta6
        g123 = twist_exp(Xi(:,1), th1) * twist_exp(Xi(:,2), th2) *
twist_exp(Xi(:,3), th3);
        g456 = g123 \ g1;
        R456 = g456(1:3, 1:3);

        % 腕部翻转/不翻转的 2 组解
        for flip = [1, -1]
            th5 = atan2(flip * sqrt(R456(1,3)^2 + R456(2,3)^2),
R456(3,3));

            if abs(sin(th5)) > 1e-4
                if flip == 1
                    th4 = atan2(R456(2,3), R456(1,3));
                    th6 = atan2(R456(3,2), -R456(3,1));
                else
                    th4 = atan2(-R456(2,3), -R456(1,3));
                    th6 = atan2(-R456(3,2), R456(3,1));
                end
            else
                th4 = 0;
                if R456(3,3) > 0
```

```
                    th6 = atan2(R456(2,1), R456(1,1));
                else
                    th6 = -atan2(R456(2,1), R456(1,1));
                end
            end

            % 将当前这组解拼接到结果矩阵中
            sol = [th1; th2; th3; th4; th5; th6];
            theta_sols = [theta_sols, sol];
        end
    end
end

% 辅助函数：旋量指数映射
function T = twist_exp(xi, theta)
    v = xi(1:3); w = xi(4:6);
    if norm(w) < 1e-6
        T = [eye(3), v * theta; 0 0 0 1];
    else
        w_hat = [0, -w(3), w(2); w(3), 0, -w(1); -w(2), w(1), 0];
        R = eye(3) + w_hat * sin(theta) + w_hat^2 * (1 - cos(theta));
        p = (eye(3) - R) * cross(w, v) + w * (w' * v) * theta;
        T = [R, p; 0 0 0 1];
    end
end
```

---

2. 六轴机器人所有 8 组解（若有）的模型截图

2.1 仿真验证主程序代码

为验证上述正逆运动学算法，编写了环境初始化脚本 `Initialization.m` 及验证主程序 `Virification.m`。

初始化环境代码（定义连杆参数与旋量）：

```
% ========================================
% Initialization.m: 环境与参数初始化
% ========================================
% 1. 定义连杆长度 (单位: mm)
L1 = 491; L2 = 450; L3 = 450; L4 = 84;
config = [L1, L2, L3, L4];

% 2. 定义初始位姿 g0 (零位状态)
g0 = [ -1,  0,  0,    0;
        0, -1,  0,    0;
```

```
        0,  0,  1, 1475;
        0,  0,  0,    1];

% 3. 定义运动旋量 Xi
Xi = [ 0,   -L1,  -(L1+L2),  0,  -(L1+L2+L3),  0;
       0,    0,     0,       0,    0,          0;
       0,    0,     0,       0,    0,          0;
       0,    0,     0,       0,    0,          0;
       0,    1,     1,       0,    1,          0;
       1,    0,     0,       1,    0,          1];
```

正逆解验证主程序代码：

```
% ========================================
% Virification.m：正逆运动学验证
% ========================================
% 调用初始化参数
Initialization;

% 自选一组测试角度（单位：rad）
theta_test = [pi/3, pi/4, pi/2, pi/6, pi/3, pi/6];

% 1. 调用正解函数验证末端位姿
T_calc = Fkine(Xi, theta_test, g0);
fprintf('Matlab 计算的末端位姿矩阵：\n');
disp(T_calc);

% 2. 使用目标位姿，求取全部 8 组逆解
target_T = T_calc;
theta_sols_all = Ikine6s_all(target_T, config);
fprintf('求得有效逆解组数：%d\n', size(theta_sols_all, 2));
disp(theta_sols_all);

% 3. 求取单组优选逆解
theta_sols = Ikine6s(target_T, config);
fprintf('选取有效逆解：\n');
disp(theta_sols);
```

2.2 多重解的 Simulink 仿真模型截图

运行上述主程序后，求得 8 组有效逆解。

```
>> Initialization
>> Virification
Matlab 计算的末端位姿矩阵：
    0.8719    0.1585   -0.4634  279.2734
```

```
    0.2111   -0.9754    0.0634  556.4619
   -0.4419   -0.1531   -0.8839  416.7538
         0         0         0    1.0000

求得有效逆解组数：8
    1.0472    1.0472    1.0472    1.0472   -2.0944   -2.0944   -2.0944
-2.0944
    2.3562    2.3562    0.7854    0.7854   -0.7854   -0.7854   -2.3562
-2.3562
   -1.5708   -1.5708    1.5708    1.5708   -1.5708   -1.5708    1.5708
 1.5708
    0.7137   -2.4279    0.5236   -2.6180   -2.6180    0.5236   -2.4279
 0.7137
    2.4189   -2.4189    1.0472   -1.0472    1.0472   -1.0472    2.4189
-2.4189
    1.3807   -1.7609    0.5236   -2.6180    0.5236   -2.6180    1.3807
-1.7609

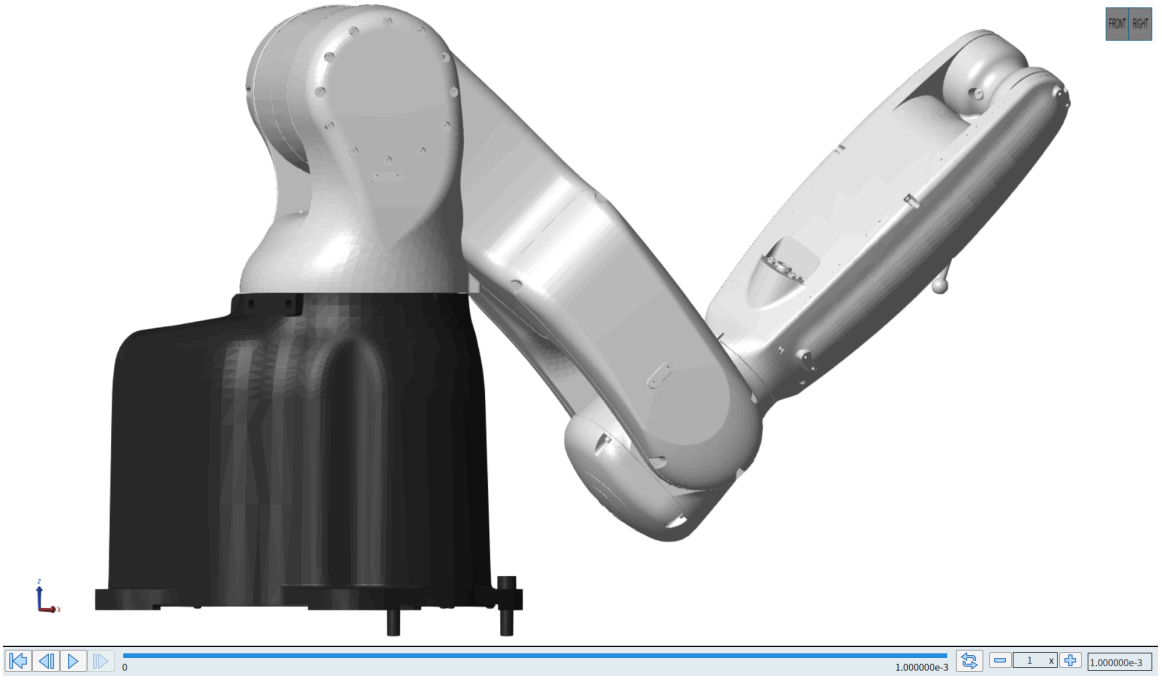
选取有效逆解：
    1.0472    2.3562   -1.5708    0.7137    2.4189    1.3807
```

将这 8 组关节角度分别输入 Simulink 仿真模型中，尽管机器人的关节姿态各不相同（涵盖左/右手、肘部上/下、腕部翻转/不翻转等 8 种构型），但其末端 TCP 位置与姿态均能准确到达目标空间点。

解 **1** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

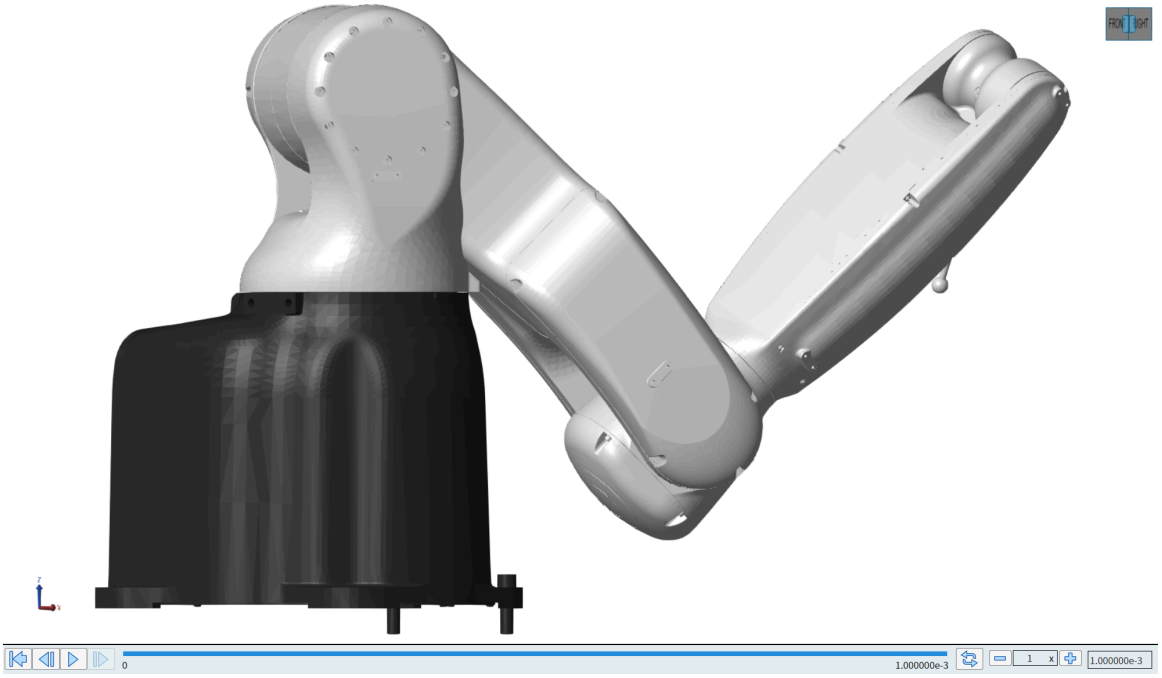
  `[1.0472,  2.3562, -1.5708,  0.7137,  2.4189,  1.3807]`



解 **2** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

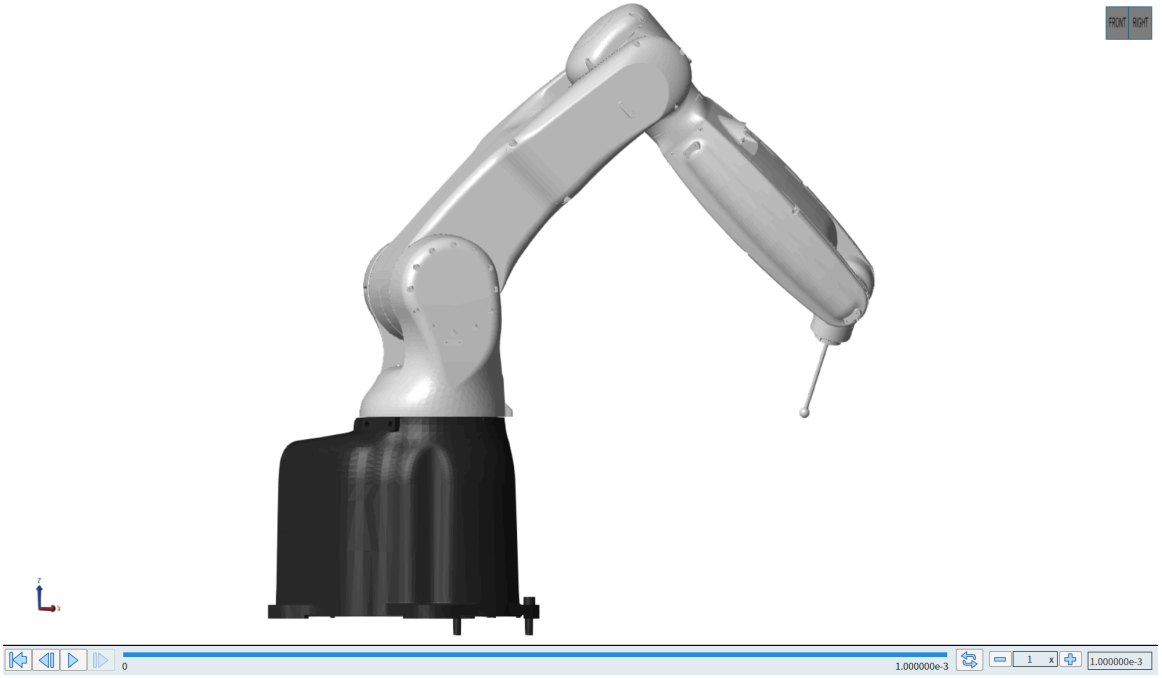
  ```
  [1.0472,  2.3562, -1.5708, -2.4279, -2.4189, -1.7609]
  ```



解 **3** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

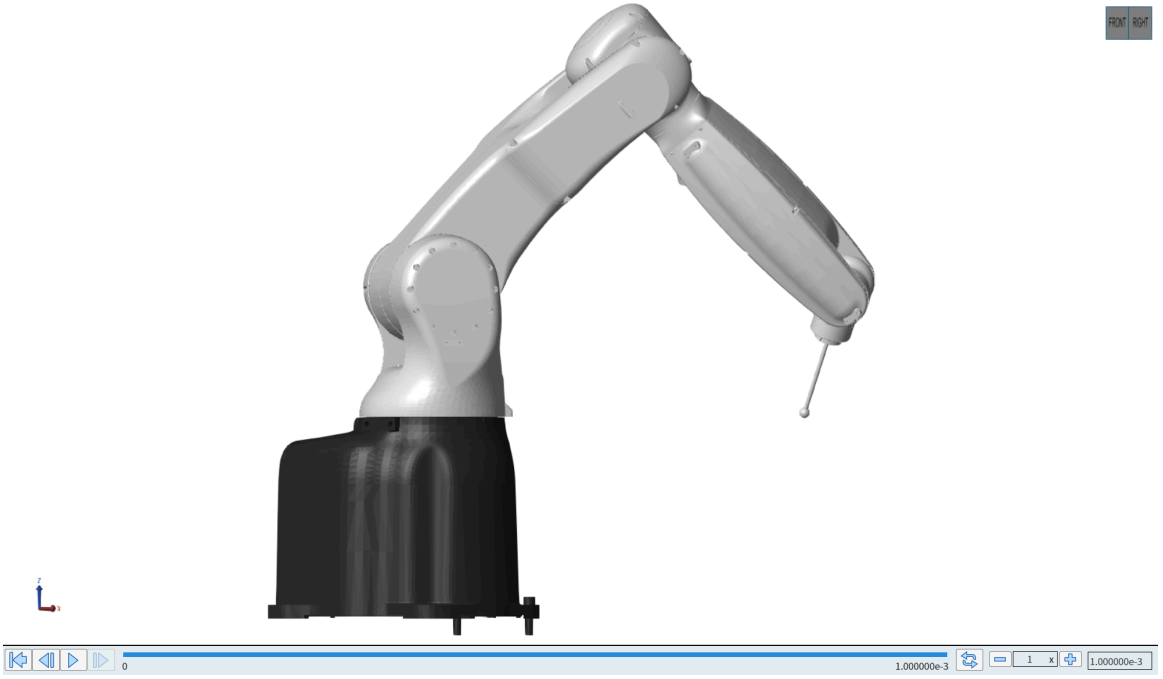
  ```
  [1.0472,  0.7854,  1.5708,  0.5236,  1.0472,  0.5236]
  ```



解 **4** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

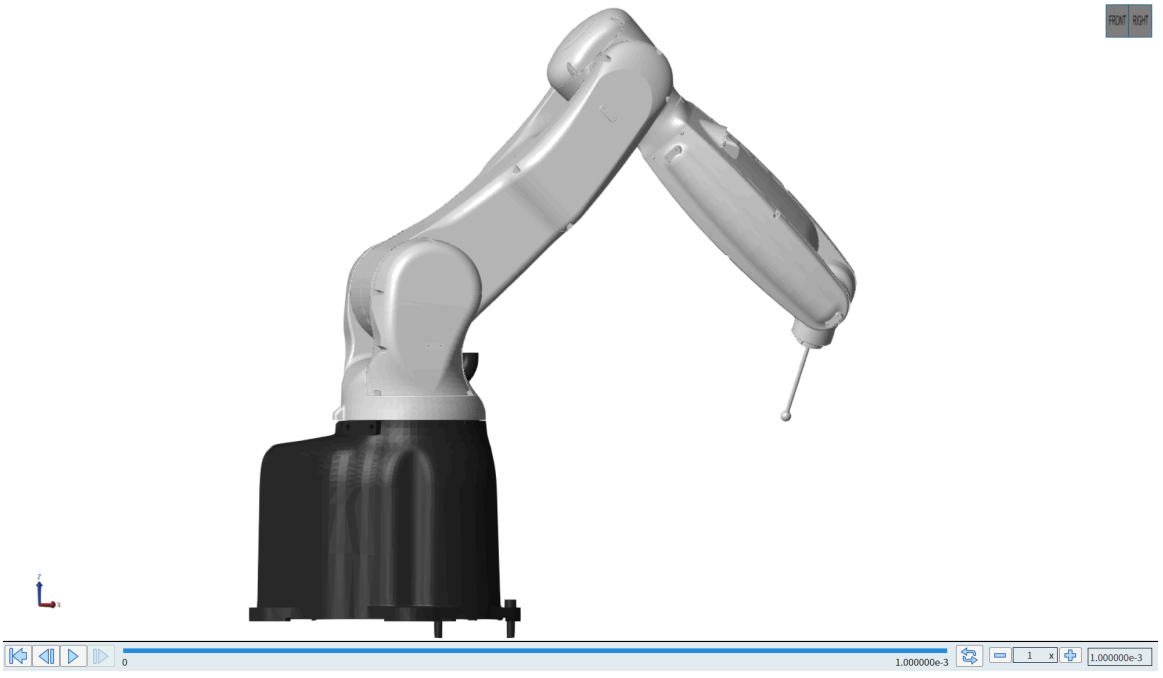
```
[1.0472,  0.7854,  1.5708, -2.6180, -1.0472, -2.6180]
```



解 **5** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

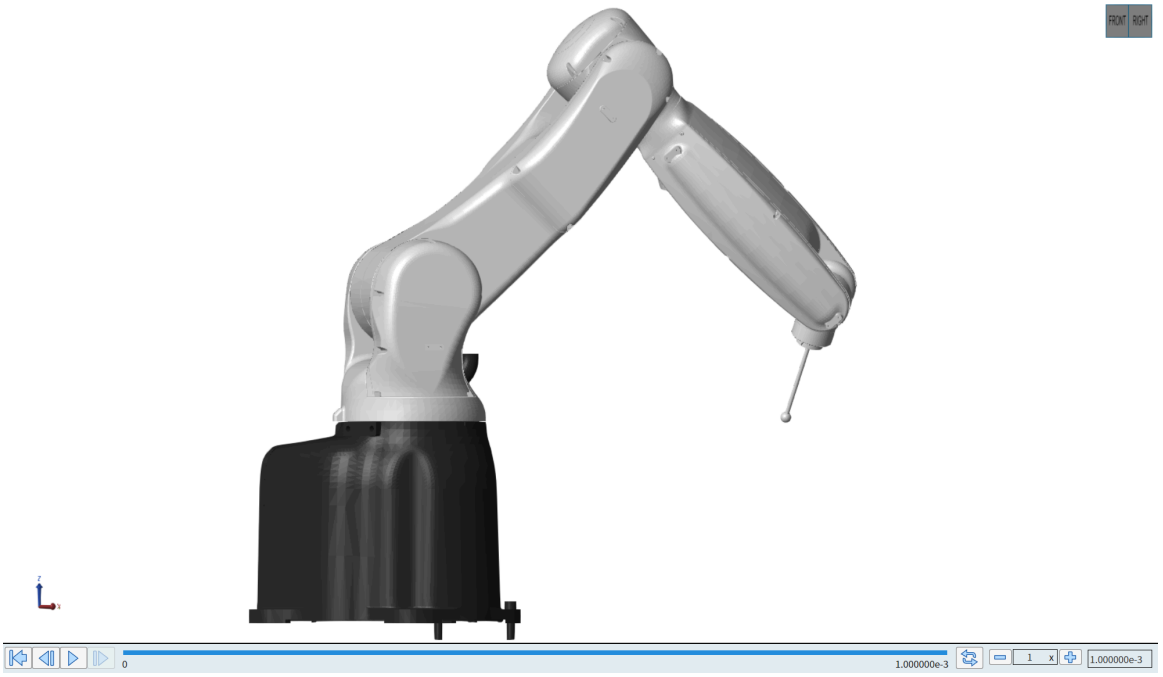
```
[-2.0944, -0.7854, -1.5708, -2.6180,  1.0472,  0.5236]
```



解 **6** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

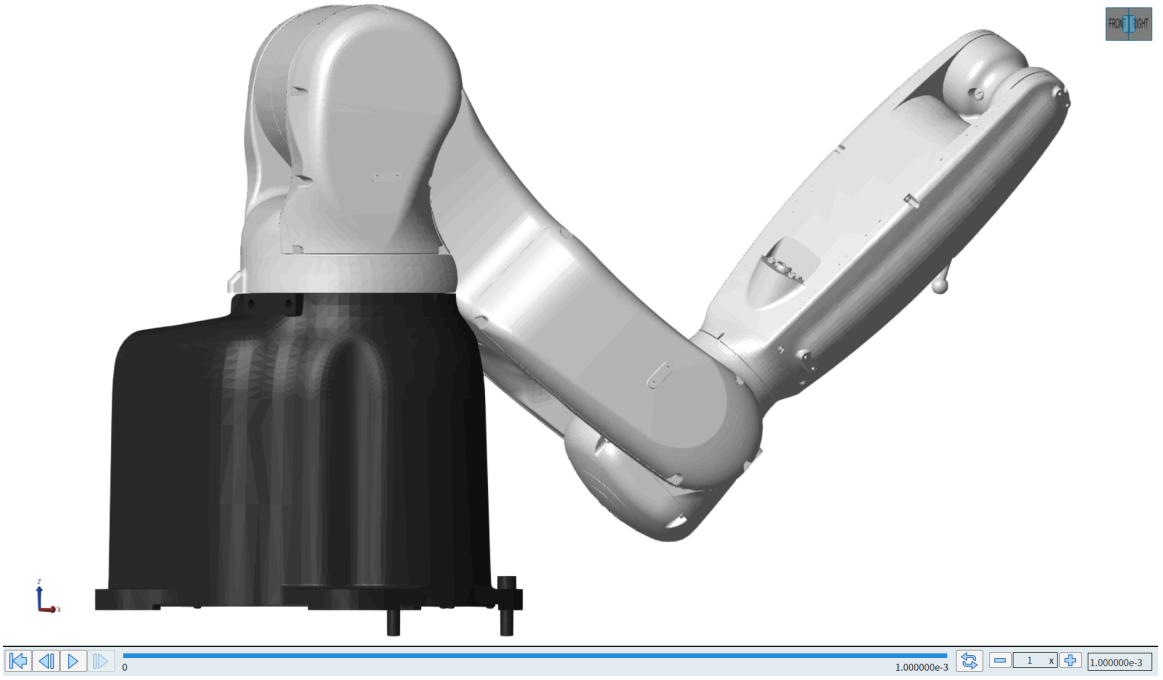
```
[-2.0944, -0.7854, -1.5708,  0.5236, -1.0472, -2.6180]
```



解 **7** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

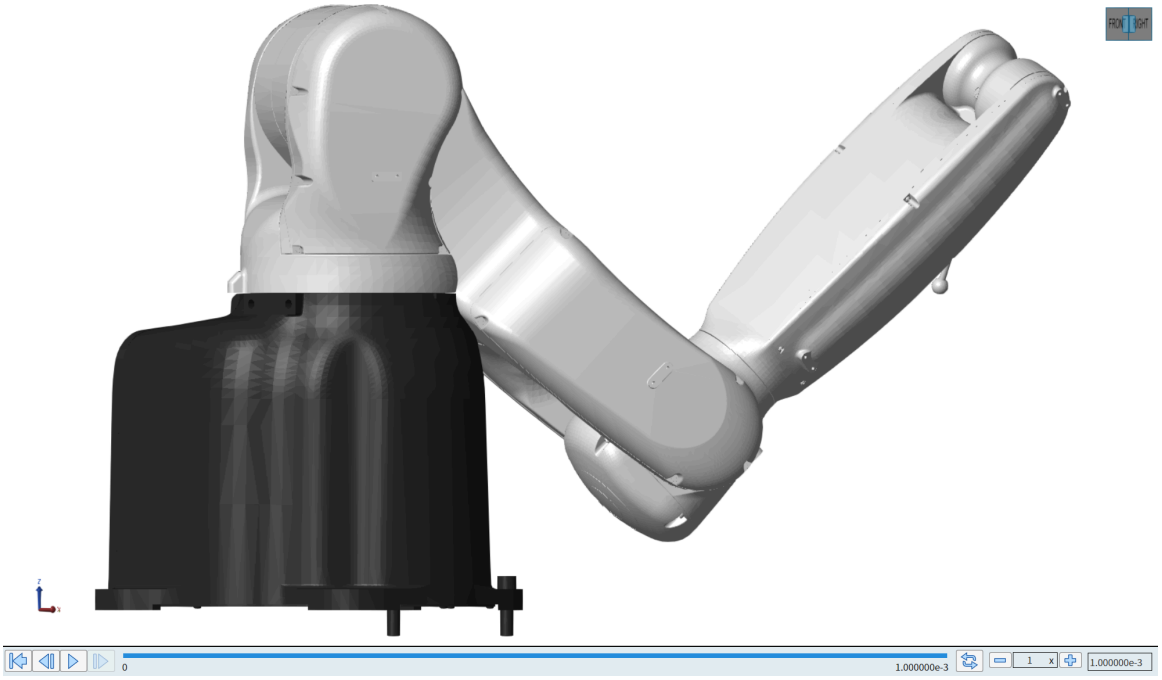
  ```
  [-2.0944, -2.3562,  1.5708, -2.4279,  2.4189,  1.3807]
  ```



解 **8** 构型

- 各关节角度（$\theta_1 \sim \theta_6$, 单位：**rad**）：

  ```
  [-2.0944, -2.3562,  1.5708,  0.7137, -2.4189, -1.7609]
  ```



---

3. 写出选取逆解的方法

在获取标准六轴工业机器人的多组逆解后，本实验主要通过工程安全规则限定法来选取唯一最优解。工业界通常优先选择最安全的默认形态，即要求机械臂尽可能远离下方工作台并保持手腕顺畅，以避免干涉碰撞。

在编写单组解函数 `Ikine6s.m` 时，本实验采用了以下特定的规则限定逻辑进行选解，并剔除了其它分支：

1. 基础选向：在子问题 1 求解 θ_1 时，仅保留正向解，剔除背向（$+180°$）解。
2. 肘部选向（优先 **Elbow Up**）：在子问题 3 求解 θ_3 时，面对一元二次方程带来的正负两个平方根，严格选取负根以确保计算出的姿态为“肘部朝上”。这能最大程度避免大臂与工作台发生干涉。
3. 腕部选向（优先无翻转）：在求解 $\theta_4, \theta_5, \theta_6$ 构成的 Z-Y-Z 欧拉角时，严格限制 θ_5 选取正角（无翻转状态），避免手腕过度扭曲。

具体选解的 **Matlab** 代码片段实现如下：

```
% ============================================================
% 逆解唯一化选取策略实现代码 (截取自 Ikine6s.m)
% ============================================================

% 1. 求解 theta1 (仅保留正向解，丢弃 th1_2 = th1_1 + pi)
px = p_target(1); py = p_target(2);
th1 = atan2(py, px);
```

```
    % 2. 求解 theta3 (通过符号限定严格选择 Elbow Up)
    % ... (前置余弦定理参数 D 计算略)
    th3 = atan2(-sqrt(1 - D^2), D); % 强制选取负根，即肘部朝上姿态

    % 3. 求解腕部欧拉角 (严格选择 theta5 为正的不翻转解)
    % ... (前置旋转矩阵 R456 计算略)
    th5 = atan2(sqrt(R456(1,3)^2 + R456(2,3)^2), R456(3,3)); % 强制 sqrt
取正值
```